

1. SALN Team 1

1.1. Application Architecture and JSON Schema

1.1.1. Members:

Batistil, Mansur
Camacho, Reuter Jan;
Filio, Ethan
Mislang, Gabriel
Rodrigo, Dom

2. ~~Nonmicroservices Architecture (Laravel's Default MVC)~~

Deprecated Repo: https://github.com/chry607/SALN_Backend

The original monolithic application used Laravel's default MVC pattern deployed on a single server. All HTTP requests were routed through **Nginx** to **PHP-FPM**, which ran one Laravel application backed by a single **PostgreSQL** database (`saln_db`). The **Controller** layer handled all business logic — email OTP generation, login/logout, inactivity checks, dashboard loading, saving entries, import, and PDF export. The **Model** layer managed user accounts, SALN form records, JSON payloads, status and edit history, and soft deletes. The **View** layer served server-rendered pages for the landing page, login flow, dashboard, and shared layout. Because every concern lived in a single codebase and database, this architecture was straightforward to develop but difficult to scale or deploy independently per feature.

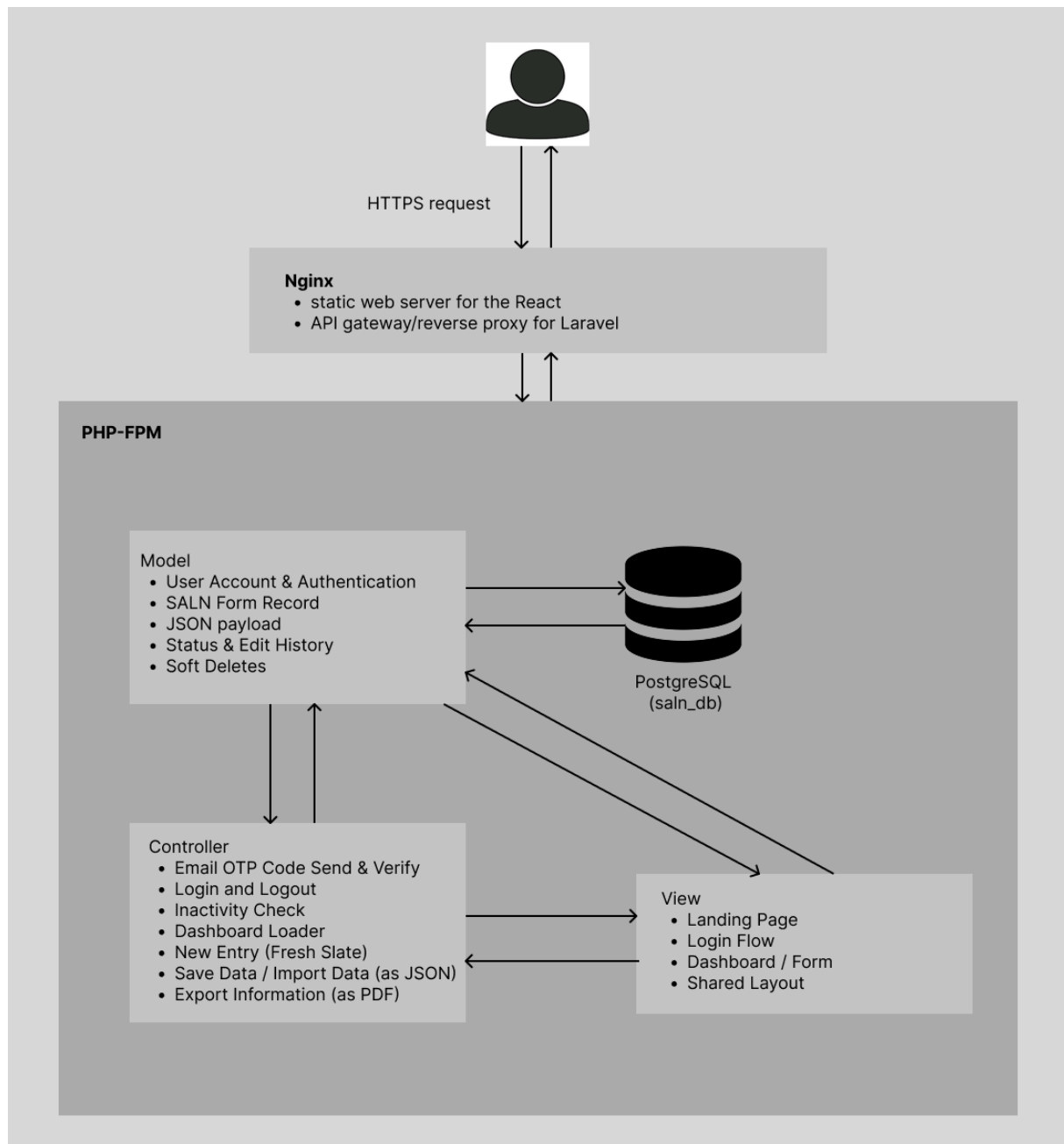


Figure 1: Nonmicroservice Architecture Diagram

3. Microservices Architecture (Digital Ocean Droplet)

Active Repo: <https://github.com/meezlung/saln-microservices/> (main branch is in **oop**)

The Digital Ocean deployment splits the application into three independent Laravel services running behind **Nginx 1.24.0** on a single Ubuntu 24.04 LTS droplet. The **React + Vite** frontend (served as static files by Nginx) communicates through `api.js` using `axios` and `react-router-dom`, with three separate base URLs targeting `/api/auth`, `/api/forms`, and `/api/documents`. Nginx acts as both a static file server for the React build and as an API gateway, rewriting paths and proxying to each Laravel service on its own port.

- **Auth API** (port 8001) — Controller handles send-code, verify-login, me, and logout. Model manages user accounts, verification codes, and login tokens (Laravel Sanctum). Database: `saln_auth_db`.
- **Forms API** (port 8002) — Controller handles save, latest, import, export, new entry, and purge. Model manages SALN form data as a JSON column. Database: `saln_form_db`. Validates Bearer tokens by calling Auth API's `/api/me` endpoint.
- **Documents API** (port 8003) — Controller handles PDF generation (via `php-pdfkit` filling form templates and merging pages) and file download. Model manages document generation records. Database: `saln_document_db`. A background queue worker (`php artisan queue:work`) processes PDF jobs asynchronously.

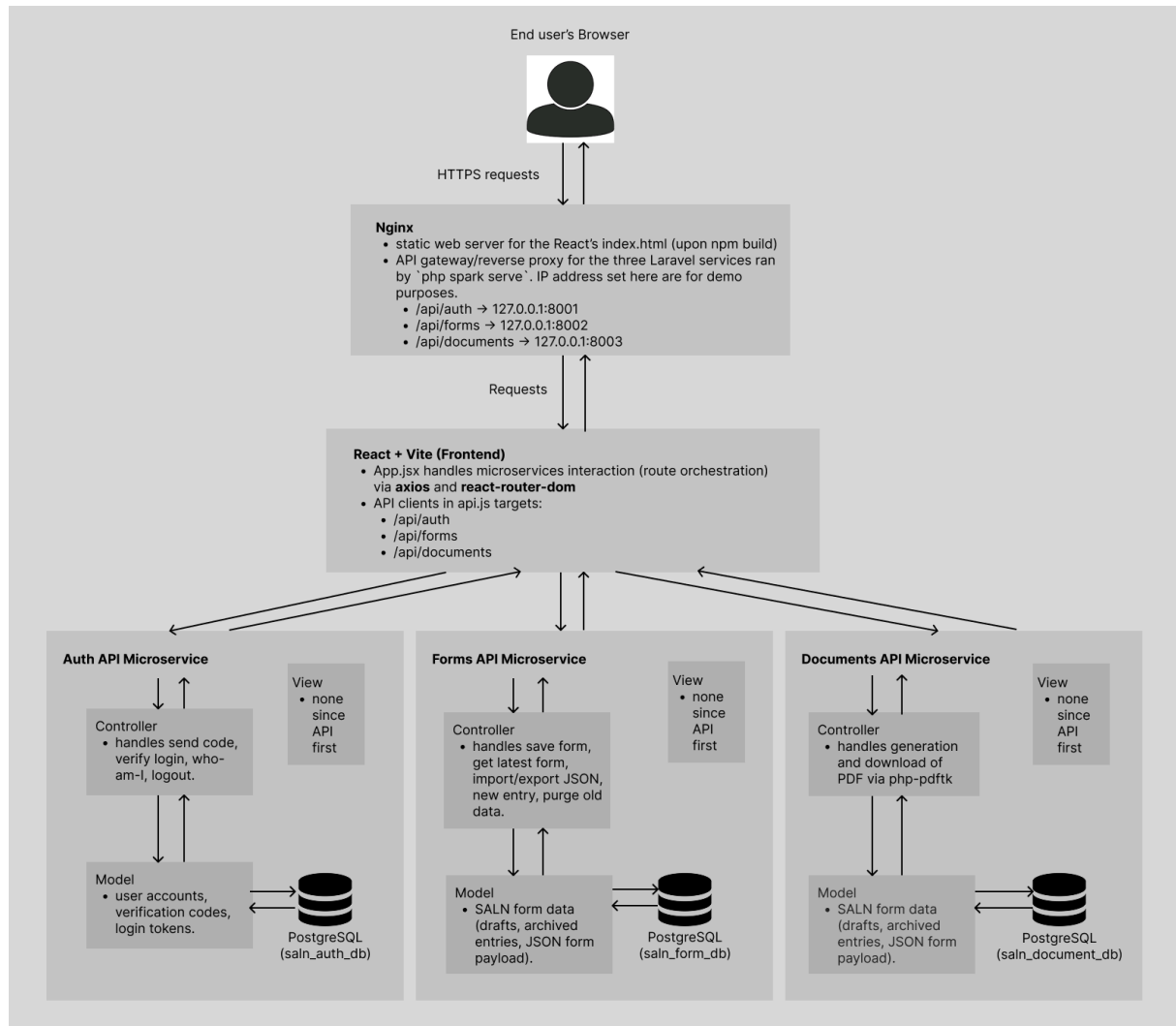


Figure 2: Microservices Architecture Diagram

4. Microservices Architecture (AWS Lambda Function)

Active Repo: <https://github.com/meezlung/saln-microservices/> (main branch is in **aws-try**)

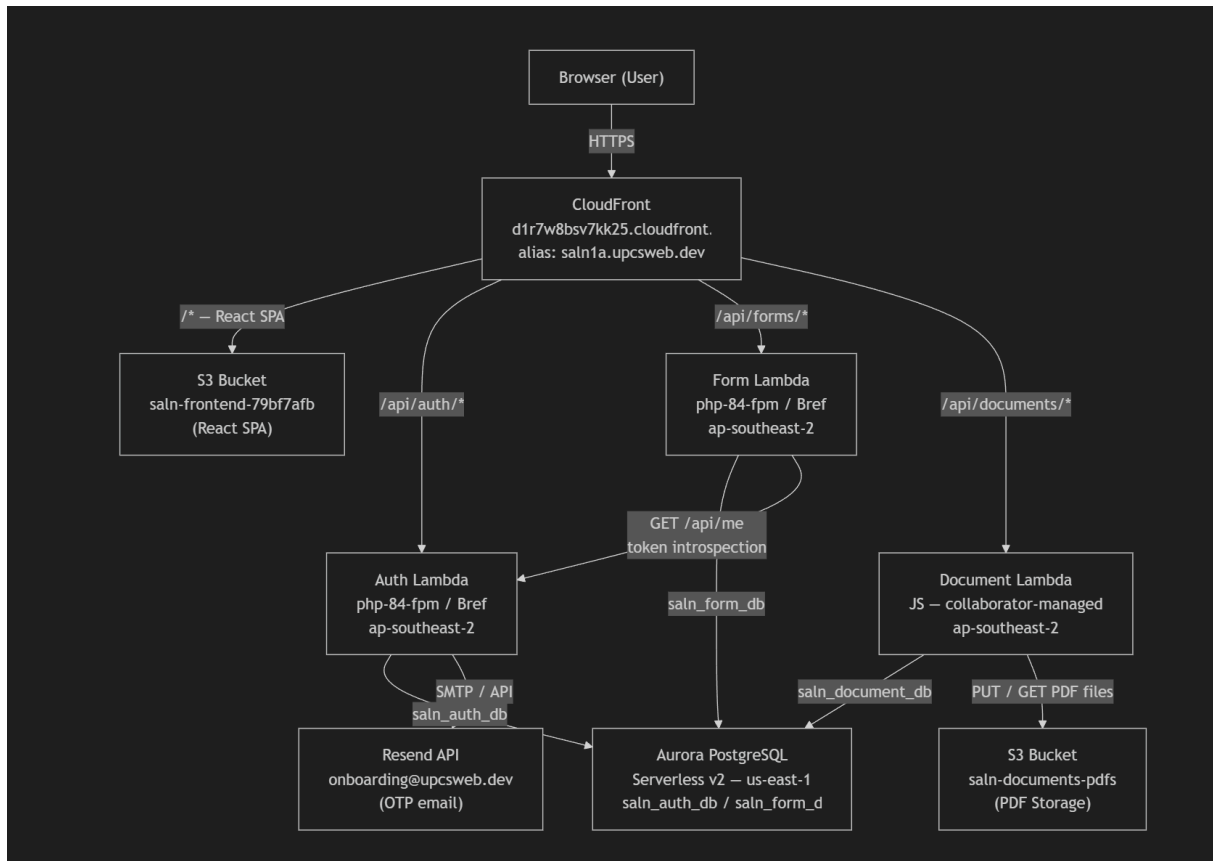


Figure 3: Microservices Architecture AWS

The AWS deployment lifts the same three Laravel microservices into serverless **AWS Lambda** functions, replacing the Digital Ocean droplet and Nginx with **Amazon CloudFront** as the CDN and API router. The React SPA is hosted in an **S3 bucket** (saln-frontend-79bf7afb) and served through CloudFront's default origin. Each PHP service runs on the **Bref** PHP-on-Lambda runtime (php-84-fpm, Amazon Linux 2023) and is deployed using **Serverless Framework v4**. The CloudFront distribution (d1r7w8bsv7kk25.cloudfront.net, aliased to saln1a.upcsweb.dev via a CNAME managed by Prof. Sir Rom Feria) routes incoming requests by path prefix to the appropriate Lambda Function URL.

CloudFront routing (in priority order):

Path Pattern	Origin	Notes
/api/auth/*	Auth Lambda Function URL	php-84-fpm, Bref
/api/forms/*	Form Lambda Function URL	php-84-fpm, Bref
/api/documents/*	Document Lambda Function URL	JS Lambda (collaborator-managed)
/api/*	saln1.upcsweb.dev (nginx fallback)	Digital Ocean droplet
/*	S3 bucket (SPA)	403/404 → index.html

Path prefix stripping: CloudFront forwards the full request path to each Lambda. Each PHP Lambda's public/index.php rewrites REQUEST_URI before Laravel boots using two environment variables: LAMBDA_PATH_PREFIX (the CloudFront prefix to match) and LAMBDA_PATH_REPLACE (what to substitute). For example, LAMBDA_PATH_PREFIX=/api/auth and LAMBDA_PATH_REPLACE=/api rewrites /api/auth/send-code → /api/send-code, which matches the Laravel route. The JS document Lambda receives the full /api/documents/* path and routes internally without stripping.

Service descriptions:

- **Auth Lambda** (saln-auth-service-dev-web) — Handles OTP email (via **Resend API** using `onboarding@upcsweb.dev`), login/logout, and token introspection (`GET /api/me`). Uses **Laravel Sanctum** for Bearer token issuance. Writes temporary files (Symfony Mailer, view cache) to `/tmp/laravel` since `/var/task` is read-only in Lambda. The `pdo_pgsql` PostgreSQL extension is loaded explicitly via `php/conf.d/pdo_pgsql.ini` and a Bref-provided Lambda layer (`pgsql-php-84:4`).
- **Form Lambda** (saln-form-service-dev-web) — Handles SALN form CRUD and export. On every authenticated request, the `AuthServiceIntrospection` middleware calls `GET /api/me` on the Auth Lambda with the Bearer token (timeout: 15 s to survive cold starts) and extracts the `user.id` UUID for downstream use.
- **Document Lambda** (collaborator-managed, JS runtime) — Handles PDF generation and download. Accepts `Authorization: Bearer <token>` and `X-User-Id: <uuid>` headers directly from the frontend. CloudFront forwards `X-User-Id` via an origin request policy whitelist. Stores generated PDFs in S3 bucket `saln-documents-pdfs` (`ap-southeast-2`).
- **Database** — Amazon Aurora PostgreSQL Serverless v2 cluster in `us-east-1` with three databases: `saln_auth_db`, `saln_form_db`, `saln_document_db`. Cross-region from the `ap-southeast-2` Lambdas; SSL required (`DB_SSLMODE=require`).

Cold start behavior: PHP Lambdas cold-start in approximately 4–6 s (PHP runtime boot + Composer autoloader + Laravel bootstrap). The Form Lambda additionally calls the Auth Lambda for token introspection on every request — if both are cold simultaneously, combined latency reaches 10–15 s. Cold starts occur after any period of inactivity. Subsequent warm requests complete in under 1 s.

Infrastructure provisioning: CloudFront distribution and S3 bucket are managed by **Terraform** (`infra/frontend-static/terraform`). The React SPA is deployed with `aws s3 sync` and a CloudFront invalidation. Lambda functions are deployed with `sudo serverless deploy` from each service directory; migrations run via `serverless bref:cli --args="migrate --force"`.

5. JSON Schema

The SALN filing system stores each form as a single JSON document, versioned at **schema v1.1.0**. The schema is defined by `createEmptyForm()` in `frontend/web/src/lib/salnForm.js`. Below is the complete structure with field descriptions.

```
{
  "schema_version": "1.1.0",

  "form_metadata": {
    "form_type": "SALN_2025",
    "compliance_type": "ASSUMPTION | ANNUAL",
    "as_of_date": "YYYY-MM-DD",
    "filing_type": "JOINT | SEPARATE | NOT_APPLICABLE"
  },

  "declarant": {
    "personal_information": {
      "last_name": "",
      "first_name": "",
```

```

        "middle_initial": "",
        "position": "",
        "agency_office": "",
        "office_address": ""
    }
},

"spouses": [
    {
        "id": "<row-uuid>",
        "is_public_official": false,
        "last_name": "",
        "first_name": "",
        "middle_initial": "",
        "position": "",
        "agency_office": "",
        "office_address": ""
    }
],

"children_below_18": [
    {
        "id": "<row-uuid>",
        "name": "",
        "birthday": "YYYY-MM-DD",
        "age": null
    }
],

"assets": {
    "declarant": {
        "real_properties": [
            {
                "description": "",
                "kind": "",
                "exact_location": "",
                "assessed_value": "",
                "fair_market_value": "",
                "acquisition": {
                    "year": "",
                    "mode": "",
                    "cost": ""
                }
            }
        ],
        "personal_properties": [
            {
                "description": "",
                "acquisition_year": "",
                "acquisition_cost": ""
            }
        ]
    },
    "spouse_children": {
        "real_properties": [ /* same structure as declarant.real_properties */ ],

```

```

        "personal_properties": [ /* same structure as declarant.personal_properties
*/ ]
    },
    "liabilities": {
        "declarant": [
            {
                "nature": "",
                "creditor_name": "",
                "outstanding_balance": ""
            }
        ],
        "spouse_children": [ /* same structure */ ]
    },
    "business_interests": {
        "declarant": {
            "has_business_interest": false,
            "entries": [
                {
                    "entity_name": "",
                    "business_address": "",
                    "nature_of_interest": "",
                    "date_acquired": "YYYY-MM-DD"
                }
            ]
        },
        "spouse_children": {
            "has_business_interest": false,
            "entries": [ /* same structure */ ]
        }
    },
    "relatives_in_government": {
        "has_relatives": false,
        "entries": [
            {
                "relative_name": "",
                "relationship": "",
                "position": "",
                "agency_office": ""
            }
        ]
    },
    "certification": {
        "date_signed": "YYYY-MM-DD | null",
        "authorization_to_verify": false
    }
}

```

Field notes:

- `schema_version` — Used by `detectSALNSchema()` to distinguish the current schema ("og") from a legacy import format ("new"). `mapSALN()` converts legacy data to the current schema on import.
- `form_metadata.compliance_type` — Either "ASSUMPTION" (first filing upon assumption of office) or "ANNUAL" (yearly filing).
- `form_metadata.filing_type` — "JOINT" includes spouse/children assets in the same form; "SEPARATE" or "NOT_APPLICABLE" limits to the declarant only.
- `assets / liabilities / business_interests` — Each is split into declarant and spouse_children scopes so joint and separate filing modes can render the correct columns.
- `spouses[].id / children_below_18[].id` — Client-generated UUIDs (via `crypto.randomUUID()`) used as React list keys and for stable row identity across save/load cycles.
- `children_below_18[].age` — Computed at load time from birthday; never persisted to the database.
- Numeric fields (`assessed_value`, `fair_market_value`, `acquisition.cost`, `acquisition_cost`, `outstanding_balance`) are stored as strings to preserve exact decimal input and avoid floating-point rounding; formatted for display with `formatCurrency()`.